

BFL on PHMRC: reproducing the AP results across three label-shift cases

Jason Teng, Yu Zhu, Zehang Richard Li

Contents

Case I: no within-target label shift	2
Baselines: local-self and local-avg	3
BFL-base: sources only, no target labels	4
BFL-domain: labels fed into a candidate model	4
BFL-partial: labels fed into the BFL model	5
BFL-mix: a split of both	5
Accuracy across methods (Case I)	5
Inspecting a fitted object	6
Case II: mild within-target label shift	10
Baselines	11
BFL fits (domain, partial, mix)	12
Accuracy across methods (Case II)	13
Case III: severe within-target label shift	13
Baselines	14
BFL fits (domain, partial, mix)	15
Accuracy across methods (Case III)	15
Additional figures (BFL-base, no within-target label shift)	16
Figure 8: estimated vs true prevalence	16
Figure 10: domain weights (lambda)	17
Session info	18

This example runs the paper’s BFL variants on the real **PHMRC** verbal-autopsy data, with **AP** (Andhra Pradesh) as the target and the other five sites as sources, across the three within-target label-shift settings from the paper:

- **Case I — no shift** (20% of AP labeled): baselines + **base, domain, partial, mix**.
- **Case II — mild shift**: baselines + **domain, partial, mix**.

- **Case III — severe shift:** baselines + domain, partial, mix.

base uses no target labels, so it is shift-invariant and only run in Case I. The point is to show how each variant is a single `run_BFL()` call followed by `score_BFL()` — the only things that change are the arguments you pass. Notice that the shift cases flip `label_shift = TRUE` and score on the unlabeled subset via `eval_idx`; everything else is the same call. The LCVA fitting and summary assembly live in `helpers.R`. Requires LCVA and BFL; the PHMRC data is bundled, so run from this folder.

```
library(LCVA)
library(BFL)
source("helpers.R")    # LCVA + PHMRC glue + per-variant input builders + baselines
```

The two **baselines** (`local_self`, `local_avg`) are LCVA models that do *not* use the BFL package; they are shown for comparison, as in the paper. Each `inputs_*(ctx)` assembles that variant's `local_summaries` (and `X_target`, `Y_target`, `Y_add`) from the LCVA phis; the package calls are written out below.

Case I: no within-target label shift

Load AP and the five sources, fit each source's local LCVA model, and split AP into 20% labeled / 80% unlabeled. Notice that this step fitting multiple LCVA models is the most time-consuming part of the example. This step can take 20 minutes or more.

```
ctx1 <- prepare_ap(target = "AP",
                   sources = c("Mexico", "Bohol", "Dar", "Pemba", "UP"),
                   seed = 1, shift = "no_shift")
#> Stacking 3 chains, with 1586 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 21.31 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 547 708 745
#> Stacking 3 chains, with 1259 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 19.6 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 755 788 457
#> Stacking 3 chains, with 1726 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 20.23 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 744 649 607
#> Stacking 3 chains, with 297 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 19.24 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
```

```

#> 419 535 1046
#> Stacking 3 chains, with 1419 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 22.1 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 700 768 532

c(N = length(ctx1$Y_new), labeled = length(ctx1$labeled),
  unlabeled = length(ctx1$missing), sources = length(ctx1$src_sites))
#>      N      labeled unlabeled  sources
#> 1554      311     1243      5

```

Baselines: local-self and local-avg

Here we consider two simple baseline cases: - *local-self*: train the model on the labeled subset of AP data only, and predict on the unlabeled AP data. - *local-avg*: train the model on each of the five sites separately, and predict on the unlabeled AP data. Then average the prediction performance.

```

b1_self <- local_self(ctx1)
#> Stacking 3 chains, with 311 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 19.24 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 287 1321 392

b1_avg <- local_avg(ctx1)
#> Stacking 3 chains, with 1586 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 20.09 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 547 708 745
#> Stacking 3 chains, with 1259 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 19.94 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 755 788 457
#> Stacking 3 chains, with 1726 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 20.33 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 744 649 607
#> Stacking 3 chains, with 297 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05

```

```

#>
#> Total elapsed time for approximate LOO and stacking = 19.16 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 419 535 1046
#> Stacking 3 chains, with 1419 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 19.43 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 700 768 532
round(rbind(local_self = b1_self, local_avg = b1_avg), 4)
#>           top1 top1bal  csmf
#> local_self 0.2936 0.1818 0.6105
#> local_avg 0.2351 0.1822 0.6285

```

BFL-base: sources only, no target labels

In the *base* case, we use the source-domain fitted model as input to BFL, and ignore the labeled data in the target site AP.

```

inp <- inputs_base(ctx1)
fit_base <- run_BFL(inp$local_summaries, X_target = inp$X_target,
                    Y_target = inp$Y_target, label_shift = ctx1$label_shift,
                    sampler = "gibbs")
sc_base <- score_BFL(fit_base, Y_eval = inp$Y_eval, eval_idx = inp$eval_idx)
c(top1 = sc_base$top1_acc, top1bal = sc_base$balanced_acc, csmf = sc_base$csmf_acc)
#>           top1 top1bal  csmf
#> 0.2940798 0.2363677 0.5563760

```

BFL-domain: labels fed into a candidate model

In the *BFL-domain* case, we use the fitted models based on each source domain and the local labeled data in AP, as candidate models for BFL. That is, the labeled rows train a self-model (added to `local_summaries`).

```

inp <- inputs_domain(ctx1)
#> Stacking 3 chains, with 311 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 19.18 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 287 1321 392
fit_dom <- run_BFL(inp$local_summaries, X_target = inp$X_target,
                    Y_target = inp$Y_target, Y_add = inp$Y_add,
                    label_shift = ctx1$label_shift, sampler = "gibbs")
sc_dom <- score_BFL(fit_dom, Y_eval = inp$Y_eval, eval_idx = inp$eval_idx)
c(top1 = sc_dom$top1_acc, top1bal = sc_dom$balanced_acc, csmf = sc_dom$csmf_acc)
#>           top1 top1bal  csmf
#> 0.3226066 0.2541346 0.7172464

```

BFL-partial: labels fed into the BFL model

In the *BFL-domain* case, we use the fitted models based on each source domain as candidate models for BFL. In addition, we treat the observed labels as partially known labels in fitting the BFL step. They go straight into `Y_target` (NA on the unlabeled rows).

```
inp <- inputs_partial(ctx1)
fit_par <- run_BFL(inp$local_summaries, X_target = inp$X_target,
                  Y_target = inp$Y_target, label_shift = ctx1$label_shift,
                  sampler = "gibbs")
sc_par <- score_BFL(fit_par, Y_eval = inp$Y_eval, eval_idx = inp$eval_idx)
c(top1 = sc_par$top1_acc, top1bal = sc_par$balanced_acc, csmf = sc_par$csmf_acc)
#>      top1    top1bal      csmf
#> 0.3097345 0.2607113 0.6912915
```

BFL-mix: a split of both

Half the labeled rows (per cause) train the self-model; the other half enter as partial labels. Both `Y_target` and `Y_add` are supplied.

```
inp <- inputs_mix(ctx1)
#> Stacking 3 chains, with 152 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 19.39 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 403 223 1374
fit_mix <- run_BFL(inp$local_summaries, X_target = inp$X_target,
                  Y_target = inp$Y_target, Y_add = inp$Y_add,
                  label_shift = ctx1$label_shift, sampler = "gibbs")
sc_mix <- score_BFL(fit_mix, Y_eval = inp$Y_eval, eval_idx = inp$eval_idx)
c(top1 = sc_mix$top1_acc, top1bal = sc_mix$balanced_acc, csmf = sc_mix$csmf_acc)
#>      top1    top1bal      csmf
#> 0.3024940 0.2559061 0.6792364
```

Accuracy across methods (Case I)

```
round(rbind(
  local_self = b1_self,
  local_avg  = b1_avg,
  base       = c(top1 = sc_base$top1_acc, top1bal = sc_base$balanced_acc, csmf = sc_base$csmf_acc),
  domain     = c(top1 = sc_dom$top1_acc, top1bal = sc_dom$balanced_acc, csmf = sc_dom$csmf_acc),
  partial    = c(top1 = sc_par$top1_acc, top1bal = sc_par$balanced_acc, csmf = sc_par$csmf_acc),
  mix        = c(top1 = sc_mix$top1_acc, top1bal = sc_mix$balanced_acc, csmf = sc_mix$csmf_acc)
), 4)
#>      top1    top1bal      csmf
#> local_self 0.2936 0.1818 0.6105
#> local_avg  0.2351 0.1822 0.6285
#> base       0.2941 0.2364 0.5564
```

```
#> domain      0.3226  0.2541 0.7172
#> partial      0.3097  0.2607 0.6913
#> mix          0.3025  0.2559 0.6792
```

Inspecting a fitted object

Every `run_BFL()` fit has class `BFL` and every `score_BFL()` result has class `BFL_score`, each with `print` / `summary` / `plot`. Shown here for `domain`.

```
fit_dom
#> BFL Fit
#>
#>   Records   : 1243
#>   Causes    : 34
#>   Models    : 6 (Mexico, Bohol, Dar, Pemba, UP, AP)
#>   Post draws: 4000
#>   Variant   : no partial labels, CSMF correction
#>
#>   Top causes: 3 (18.8%), 19 (11.1%), 15 (9.3%)
#>   Run summary() for full CSMF table.
```

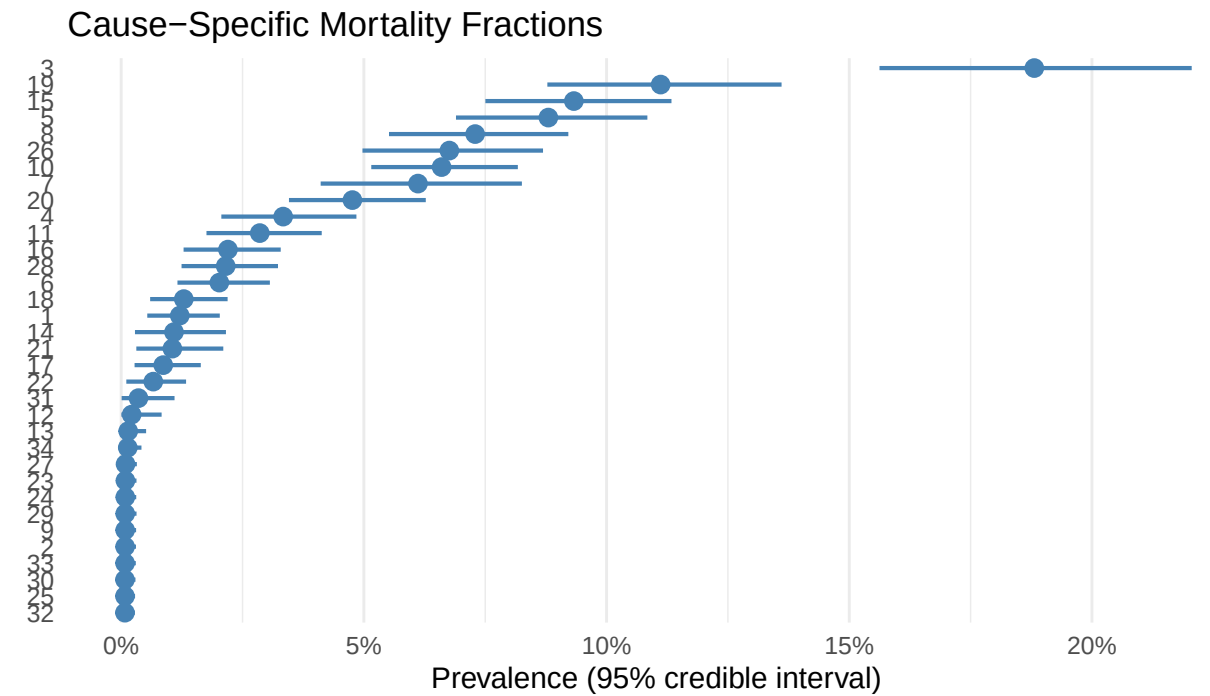
```
summary(fit_dom)
#> BFL Results
#>
#>   Records: 1243 | Models: 6 | Causes: 34 | Draws: 4000
#>
#>
#> Cause-Specific Mortality Fractions (95% CI):
#>   Cause              Mean   Lower   Upper
#>   -----
#>   3                   18.8%   15.6%   22.1%
#>   19                  11.1%    8.8%   13.6%
#>   15                   9.3%    7.5%   11.3%
#>   5                    8.8%    6.9%   10.8%
#>   8                    7.3%    5.5%    9.2%
#>   26                   6.8%    5.0%    8.7%
#>   10                   6.6%    5.2%    8.2%
#>   7                    6.1%    4.1%    8.3%
#>   20                   4.8%    3.5%    6.3%
#>   4                    3.3%    2.1%    4.8%
#>   11                   2.9%    1.8%    4.1%
#>   16                   2.2%    1.3%    3.3%
#>   28                   2.2%    1.2%    3.2%
#>   6                    2.0%    1.2%    3.1%
#>   18                   1.3%    0.6%    2.2%
#>   1                    1.2%    0.5%    2.0%
#>   14                   1.1%    0.3%    2.2%
#>   21                   1.1%    0.3%    2.1%
#>   17                   0.9%    0.3%    1.6%
#>   22                   0.7%    0.1%    1.3%
#>   31                   0.4%    0.0%    1.1%
#>   12                   0.2%    0.0%    0.8%
```

```

#> 13          0.1%    0.0%    0.5%
#> 34          0.1%    0.0%    0.4%
#> 27          0.1%    0.0%    0.3%
#> 23          0.1%    0.0%    0.3%
#> 24          0.1%    0.0%    0.3%
#> 29          0.1%    0.0%    0.3%
#> 9           0.1%    0.0%    0.3%
#> 2           0.1%    0.0%    0.3%
#> 33          0.1%    0.0%    0.3%
#> 30          0.1%    0.0%    0.3%
#> 25          0.1%    0.0%    0.3%
#> 32          0.1%    0.0%    0.3%
#>
#> Model Weights (posterior mean lambda):
#> Cause          Mexico    Bohol    Dar    Pemba    UP    AP
#> -----
#> 3          0.010    0.005    0.005    0.006    0.967    0.006
#> 19         0.010    0.008    0.007    0.007    0.962    0.007
#> 15         0.016    0.018    0.018    0.000    0.622    0.325
#> 5          0.011    0.009    0.009    0.000    0.749    0.222
#> 8          0.028    0.000    0.011    0.000    0.083    0.879
#> 26         0.011    0.012    0.011    0.019    0.038    0.908
#> 10         0.011    0.012    0.017    0.069    0.864    0.026
#> 7          0.029    0.021    0.013    0.000    0.893    0.043
#> 20         0.015    0.018    0.016    0.015    0.031    0.905
#> 4          0.023    0.022    0.021    0.000    0.911    0.022
#> 11         0.000    0.025    0.025    0.616    0.282    0.052
#> 16         0.115    0.147    0.084    0.037    0.581    0.036
#> 28         0.321    0.035    0.082    0.032    0.490    0.040
#> 6          0.068    0.033    0.042    0.032    0.790    0.034
#> 18         0.167    0.095    0.050    0.167    0.191    0.331
#> 1          0.324    0.053    0.054    0.000    0.510    0.058
#> 14         0.103    0.063    0.056    0.590    0.130    0.058
#> 21         0.081    0.059    0.064    0.059    0.398    0.339
#> 17         0.075    0.075    0.000    0.000    0.767    0.083
#> 22         0.077    0.078    0.081    0.072    0.585    0.106
#> 31         0.114    0.118    0.115    0.118    0.418    0.117
#> 12         0.178    0.180    0.165    0.000    0.301    0.176
#> 13         0.201    0.156    0.154    0.161    0.160    0.168
#> 34         0.359    0.211    0.216    0.000    0.000    0.213
#> 27         0.268    0.249    0.243    0.000    0.000    0.240
#> 23         0.216    0.194    0.199    0.000    0.194    0.197
#> 24         0.000    0.000    0.244    0.249    0.258    0.250
#> 29         0.208    0.198    0.202    0.000    0.199    0.193
#> 9          0.203    0.203    0.201    0.000    0.196    0.197
#> 2          0.200    0.199    0.199    0.000    0.197    0.205
#> 33         0.000    0.339    0.330    0.000    0.000    0.331
#> 30         0.201    0.199    0.200    0.000    0.203    0.197
#> 25         0.165    0.164    0.165    0.170    0.169    0.167
#> 32         0.250    0.245    0.258    0.000    0.000    0.247
#>
#> Note: CSMF correction applied (Stan saw a subset of records).

```

```
plot(fit_dom)
```



```
sc_dom
```

```
#> BFL Score
```

```
#>
```

```
#> Top-1 accuracy : 32.3%
```

```
#> Balanced acc. : 25.4%
```

```
#> CSMF accuracy : 71.7%
```

```
#>
```

```
#> Run summary() for per-cause detail, plot() for visuals.
```

```
summary(sc_dom)
```

```
#> BFL Score
```

```
#>
```

```
#> Top-1 accuracy : 32.3% Balanced: 25.4% CSMF: 71.7%
```

```
#>
```

```
#>
```

```
#> Per-Cause Detail:
```

```
#> Cause TruePrev EstPrev CSMFerr Recall
```

```
#> -----
```

```
#> 8 8.8% 7.2% -1.6% 40.9%
```

```
#> 15 8.0% 9.2% +1.2% 66.3%
```

```
#> 7 6.6% 6.2% -0.3% 11.1%
```

```
#> 5 6.5% 8.5% +2.0% 61.5%
```

```
#> 20 6.5% 4.8% -1.7% 38.8%
```

```
#> 13 5.8% 1.1% -4.7% 0.0%
```

```
#> 18 5.7% 2.3% -3.4% 10.1%
```

```
#> 26 5.0% 7.0% +2.0% 30.2%
```

```
#> 12 4.9% 0.9% -4.0% 0.0%
```

```
#> 10 4.6% 6.1% +1.5% 79.3%
```

```
#> 3 3.5% 15.6% +12.1% 30.4%
```

```

#> 1 3.3% 1.5% -1.7% 21.4%
#> 21 3.2% 1.6% -1.5% 10.8%
#> 16 2.5% 2.2% -0.3% 68.8%
#> 6 2.3% 2.1% -0.2% 28.6%
#> 19 2.2% 9.0% +6.8% 30.3%
#> 11 2.1% 3.0% +0.9% 71.4%
#> 14 2.1% 1.3% -0.8% 15.4%
#> 17 2.0% 1.2% -0.8% 52.2%
#> 22 2.0% 1.2% -0.8% 42.9%
#> 28 2.0% 2.2% +0.2% 62.5%
#> 2 1.9% 0.4% -1.5% 0.0%
#> 31 1.9% 0.7% -1.2% 8.7%
#> 24 1.9% 0.3% -1.5% 0.0%
#> 4 1.6% 3.1% +1.4% 31.6%
#> 25 1.4% 0.3% -1.0% 0.0%
#> 27 0.5% 0.1% -0.3% 0.0%
#> 30 0.5% 0.1% -0.3% 0.0%
#> 9 0.3% 0.1% -0.2% 0.0%
#> 34 0.3% 0.2% -0.1% 0.0%
#> 23 0.2% 0.1% -0.1% 0.0%
#> 29 0.2% 0.1% -0.1% 0.0%
#> 32 0.1% 0.1% +0.1% 0.0%
#> 33 0.1% 0.1% +0.1% 0.0%

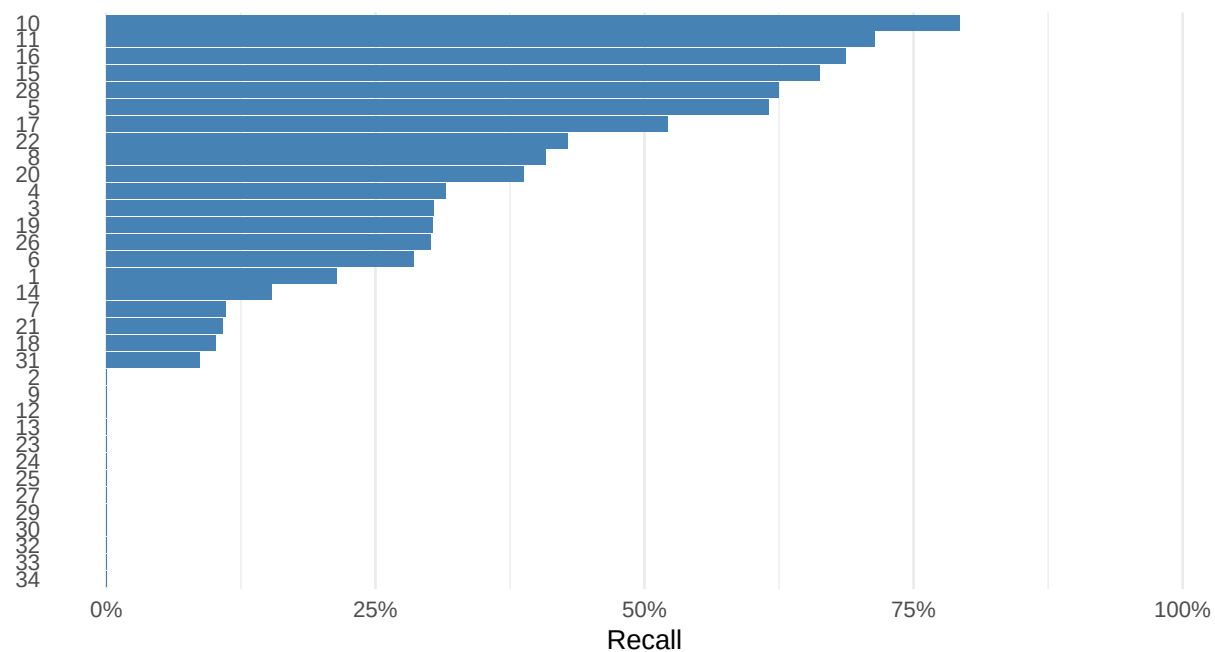
```

```

plot(sc_dom)
#> $recall

```

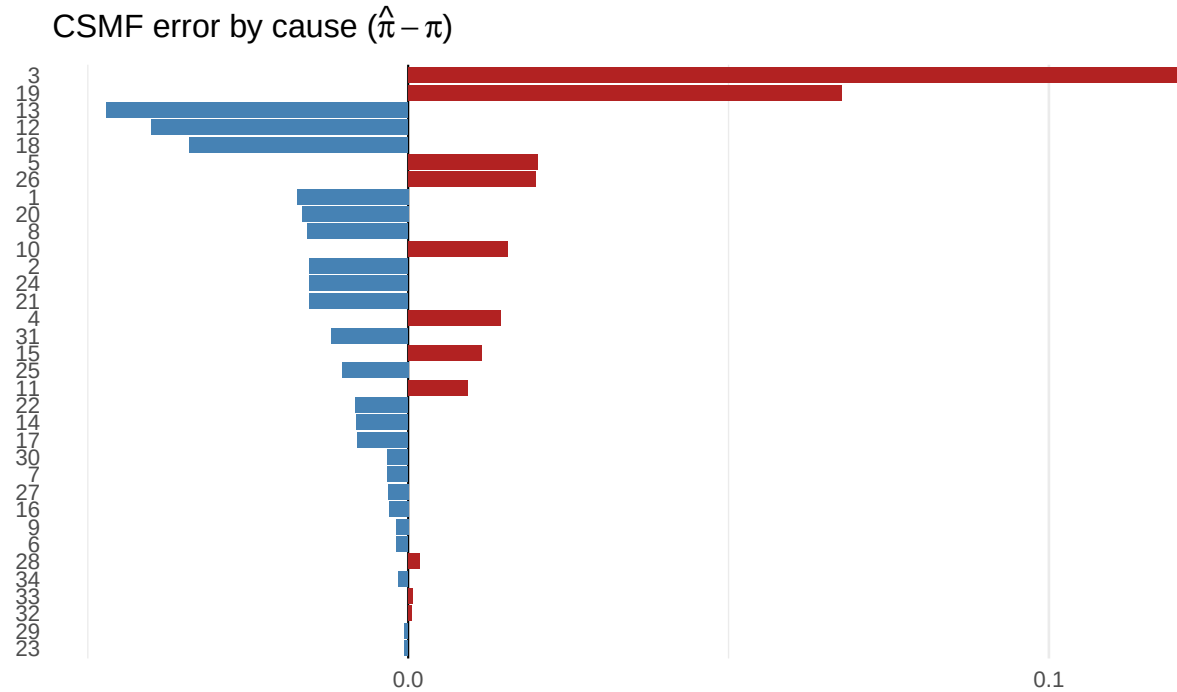
Top-1 accuracy by cause (recall)



```

#>
#> $csmf_error

```



Case II: mild within-target label shift

`prepare_ap(shift = "mild")` builds a labeled/unlabeled split with a *mild* prevalence shift between the two halves (a stacked frame drawn from independent Dirichlet prevalences). The BFL calls are identical to Case I except `label_shift = TRUE`, and scoring is on the unlabeled subset (`eval_idx`); `base` is omitted (it is shift-invariant).

```
ctx2 <- prepare_ap(target = "AP",
  sources = c("Mexico", "Bohol", "Dar", "Pemba", "UP"),
  seed = 1, shift = "mild")
#> Stacking 3 chains, with 1586 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 20.07 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 547 708 745
#> Stacking 3 chains, with 1259 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 19.85 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 755 788 457
#> Stacking 3 chains, with 1726 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 18.26 s
#> [1] "Number of posterior draws from each chain:"
```

```

#> w[1] w[2] w[3]
#> 744 649 607
#> Stacking 3 chains, with 297 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 6.95 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 419 535 1046
#> Stacking 3 chains, with 1419 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 6.41 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 700 768 532

```

Baselines

```

b2_self <- local_self(ctx2)
#> Stacking 3 chains, with 343 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 7.14 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 309 597 1094
b2_avg <- local_avg(ctx2)
#> Stacking 3 chains, with 1586 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 17.58 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 547 708 745
#> Stacking 3 chains, with 1259 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 17.29 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 755 788 457
#> Stacking 3 chains, with 1726 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 17.83 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 744 649 607
#> Stacking 3 chains, with 297 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>

```

```

#> Total elapsed time for approximate LOO and stacking = 15.9 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 419 535 1046
#> Stacking 3 chains, with 1419 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 17.45 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 700 768 532
round(rbind(local_self = b2_self, local_avg = b2_avg), 4)
#>               top1 top1bal  csmf
#> local_self 0.2357 0.2863 0.3791
#> local_avg 0.1765 0.1967 0.4532

```

BFL fits (domain, partial, mix)

```

inp      <- inputs_domain(ctx2)
#> Stacking 3 chains, with 343 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 2.99 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 309 597 1094
sc_dom2 <- score_BFL(run_BFL(inp$local_summaries, X_target = inp$X_target,
                             Y_target = inp$Y_target, Y_add = inp$Y_add,
                             label_shift = ctx2$label_shift, sampler = "gibbs"),
                    Y_eval = inp$Y_eval, eval_idx = inp$eval_idx)

inp      <- inputs_partial(ctx2)
sc_par2 <- score_BFL(run_BFL(inp$local_summaries, X_target = inp$X_target,
                             Y_target = inp$Y_target,
                             label_shift = ctx2$label_shift, sampler = "gibbs"),
                    Y_eval = inp$Y_eval, eval_idx = inp$eval_idx)

inp      <- inputs_mix(ctx2)
#> Stacking 3 chains, with 161 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 16.9 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 539 294 1167
sc_mix2 <- score_BFL(run_BFL(inp$local_summaries, X_target = inp$X_target,
                             Y_target = inp$Y_target, Y_add = inp$Y_add,
                             label_shift = ctx2$label_shift, sampler = "gibbs"),
                    Y_eval = inp$Y_eval, eval_idx = inp$eval_idx)

```

Accuracy across methods (Case II)

```
round(rbind(
  local_self = b2_self,
  local_avg  = b2_avg,
  domain    = c(top1 = sc_dom2$top1_acc, top1bal = sc_dom2$balanced_acc, csmf = sc_dom2$csmf_acc),
  partial    = c(top1 = sc_par2$top1_acc, top1bal = sc_par2$balanced_acc, csmf = sc_par2$csmf_acc),
  mix       = c(top1 = sc_mix2$top1_acc, top1bal = sc_mix2$balanced_acc, csmf = sc_mix2$csmf_acc)
), 4)
#>           top1 top1bal  csmf
#> local_self 0.2357 0.2863 0.3791
#> local_avg  0.1765 0.1967 0.4532
#> domain     0.3403 0.3729 0.4876
#> partial    0.2293 0.2683 0.3828
#> mix        0.2711 0.2964 0.4070
```

Case III: severe within-target label shift

`prepare_ap(shift = "severe")` draws a *severe* shift (per-cause Beta(0.2, 0.2) unlabeled fractions, so some causes are almost entirely unlabeled). The calls are again identical except the split, and `domain/mix` place the target self-model by its self-generated (new-space) causes — both handled inside the input builders, so the `run_BFL()/score_BFL()` calls do not change.

```
ctx3 <- prepare_ap(target = "AP",
  sources = c("Mexico", "Bohol", "Dar", "Pemba", "UP"),
  seed = 1, shift = "severe")
#> Stacking 3 chains, with 1586 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 17.68 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 547 708 745
#> Stacking 3 chains, with 1259 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 17.06 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 755 788 457
#> Stacking 3 chains, with 1726 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 19.11 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 744 649 607
#> Stacking 3 chains, with 297 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 17.47 s
```

```

#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 419 535 1046
#> Stacking 3 chains, with 1419 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 18.55 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 700 768 532

```

Baselines

```

b3_self <- local_self(ctx3)
#> Stacking 3 chains, with 704 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 18.32 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 743 873 384
b3_avg <- local_avg(ctx3)
#> Stacking 3 chains, with 1586 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 18.01 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 547 708 745
#> Stacking 3 chains, with 1259 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 18.12 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 755 788 457
#> Stacking 3 chains, with 1726 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 18.4 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 744 649 607
#> Stacking 3 chains, with 297 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 16.77 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 419 535 1046
#> Stacking 3 chains, with 1419 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05

```

```

#>
#> Total elapsed time for approximate LOO and stacking = 17.86 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 700 768 532
round(rbind(local_self = b3_self, local_avg = b3_avg), 4)
#>           top1 top1bal  csmf
#> local_self 0.0400 0.1315 0.0880
#> local_avg 0.1633 0.1815 0.3165

```

BFL fits (domain, partial, mix)

```

inp    <- inputs_domain(ctx3)
#> Stacking 3 chains, with 704 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 17.35 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 743 873 384
sc_dom3 <- score_BFL(run_BFL(inp$local_summaries, X_target = inp$X_target,
                             Y_target = inp$Y_target, Y_add = inp$Y_add,
                             label_shift = ctx3$label_shift, sampler = "gibbs"),
                    Y_eval = inp$Y_eval, eval_idx = inp$eval_idx)

inp    <- inputs_partial(ctx3)
sc_par3 <- score_BFL(run_BFL(inp$local_summaries, X_target = inp$X_target,
                             Y_target = inp$Y_target,
                             label_shift = ctx3$label_shift, sampler = "gibbs"),
                    Y_eval = inp$Y_eval, eval_idx = inp$eval_idx)

inp    <- inputs_mix(ctx3)
#> Stacking 3 chains, with 346 data points and 1500 posterior draws;
#> using stan optimizer, max iterations = 1e+05
#>
#> Total elapsed time for approximate LOO and stacking = 17.25 s
#> [1] "Number of posterior draws from each chain:"
#> w[1] w[2] w[3]
#> 0 759 1241
sc_mix3 <- score_BFL(run_BFL(inp$local_summaries, X_target = inp$X_target,
                             Y_target = inp$Y_target, Y_add = inp$Y_add,
                             label_shift = ctx3$label_shift, sampler = "gibbs"),
                    Y_eval = inp$Y_eval, eval_idx = inp$eval_idx)

```

Accuracy across methods (Case III)

```

round(rbind(
  local_self = b3_self,
  local_avg  = b3_avg,

```

```

domain = c(top1 = sc_dom3$top1_acc, top1bal = sc_dom3$balanced_acc, csmf = sc_dom3$csmf_acc),
partial = c(top1 = sc_par3$top1_acc, top1bal = sc_par3$balanced_acc, csmf = sc_par3$csmf_acc),
mix     = c(top1 = sc_mix3$top1_acc, top1bal = sc_mix3$balanced_acc, csmf = sc_mix3$csmf_acc)
), 4)
#>           top1 top1bal  csmf
#> local_self 0.0400  0.1315 0.0880
#> local_avg  0.1633  0.1815 0.3165
#> domain     0.1341  0.1348 0.3571
#> partial    0.2329  0.2311 0.4479
#> mix        0.1765  0.1565 0.3785

```

Additional figures (BFL-base, no within-target label shift)

These mirror figures 8 and 10 from the CHAMPS supplement, shown here for the **base** fit on AP under no shift. Both read straight off the fitted BFL object: `fit_base$pi` (posterior cause prevalence) and `fit_base$lambda` (posterior domain weights).

Figure 8: estimated vs true prevalence

Black point + 95% interval = BFL-base posterior prevalence; orange star = the true AP prevalence.

```

cc <- as.character(fit_base$causes) # pi column order
grab <- function(v, code) { i <- match(as.character(code), cc); if (is.na(i)) 0 else v[i] }
pim <- colMeans(fit_base$pi)
plo <- apply(fit_base$pi, 2, quantile, 0.025)
phi <- apply(fit_base$pi, 2, quantile, 0.975)
d8 <- data.frame(
  cause = factor(cause_names[codes], levels = cause_ord),
  mean = vapply(codes, function(x) grab(pim, x), 0),
  lo = vapply(codes, function(x) grab(plo, x), 0),
  hi = vapply(codes, function(x) grab(phi, x), 0),
  true = true_prev)
ggplot(d8, aes(mean, cause)) +
  geom_errorbarh(aes(xmin = lo, xmax = hi), height = 0.2) +
  geom_point(size = 2.4) +
  geom_point(aes(x = true), shape = 8, colour = "orange", size = 2.4, stroke = 1.1) +
  labs(x = "Prevalence", y = NULL,
    title = sprintf("BFL-base: estimated vs true prevalence (AP, no shift) --- CSMF %.3f",
      sc_base$csmf_acc)) +
  theme_bw(base_size = 12)

```

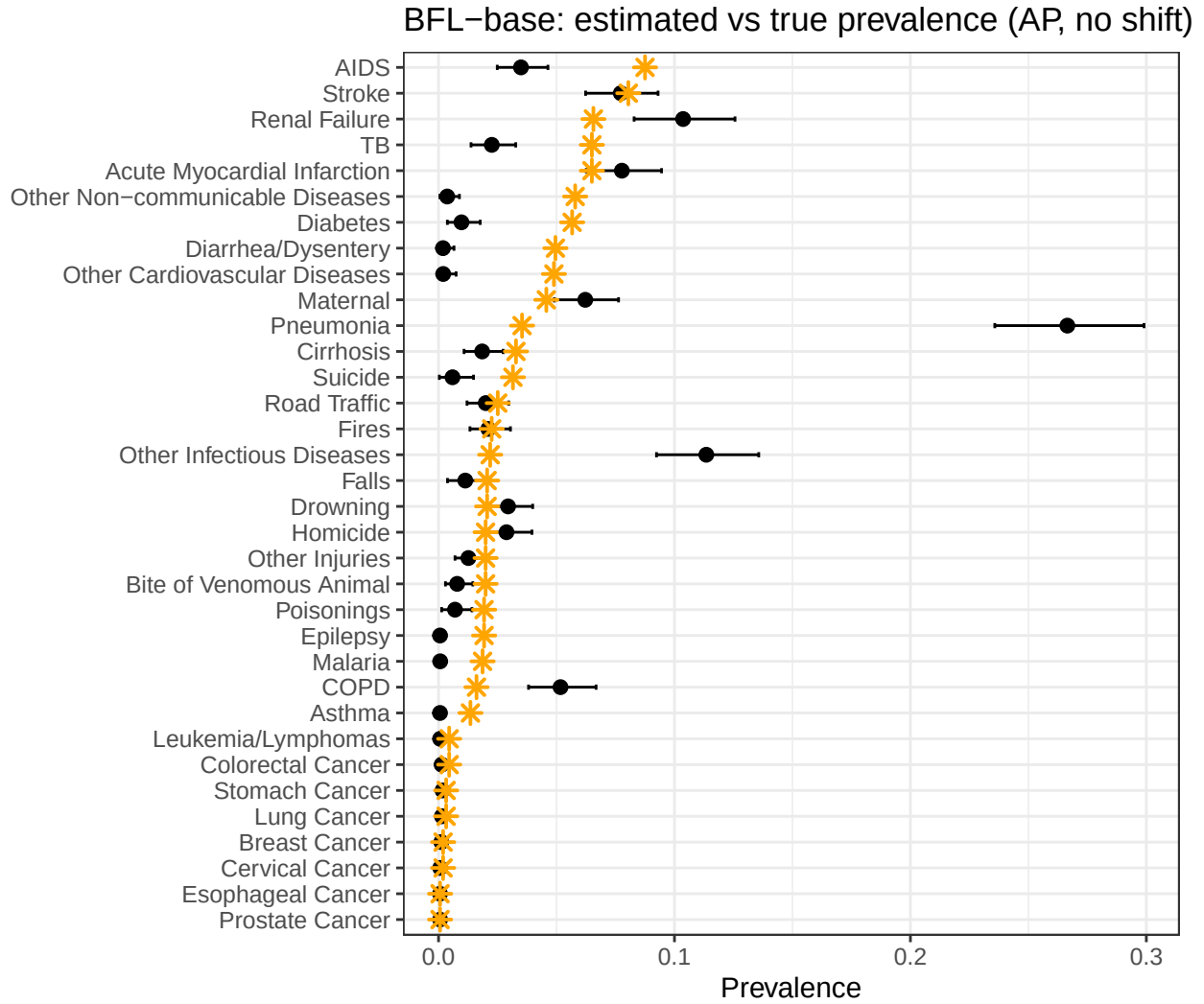


Figure 10: domain weights (lambda)

Posterior-mean weight BFL-base places on each source for each cause. This is just the package generic `plot(fit, type = "lambda")` — we relabel the numeric cause codes to PHMRC cause names and pass the prevalence order so the rows match Figure 8 (most prevalent on top).

```
f <- fit_base
f$causes <- cause_names[as.integer(f$causes)]           # origin code -> PHMRC cause name
plot(f, type = "lambda", causes = rev(cause_ord),       # rev(): most prevalent on top
     title = "BFL-base: posterior-mean domain weights (AP, no shift)")
```

BFL-base: posterior-mean domain weights (AP, no



Session info

```
sessionInfo()
#> R version 4.5.2 (2025-10-31)
#> Platform: aarch64-apple-darwin20
#> Running under: macOS Tahoe 26.1
#>
#> Matrix products: default
#> BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK v
#>
#> locale:
#> [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
#>
#> time zone: America/Los_Angeles
#> tzcode source: internal
#>
```

```

#> attached base packages:
#> [1] stats      graphics  grDevices  utils      datasets  methods    base
#>
#> other attached packages:
#> [1] ggplot2_4.0.2 BFL_2.0.0    LCVA_0.2.0
#>
#> loaded via a namespace (and not attached):
#> [1] gtable_0.3.6      dplyr_1.2.0      compiler_4.5.2
#> [4] tidyselect_1.2.1  Rcpp_1.1.2       parallel_4.5.2
#> [7] gridExtra_2.3     callr_3.7.6      scales_1.4.0
#> [10] yaml_2.3.12       fastmap_1.2.0    R6_2.6.1
#> [13] labeling_0.4.3    generics_0.1.4   knitr_1.51
#> [16] tibble_3.3.1      pillar_1.11.1    RColorBrewer_1.1-3
#> [19] rlang_1.3.0       inline_0.3.21    rstan_2.32.7
#> [22] xfun_0.56         S7_0.2.1         RcppParallel_5.1.11-1
#> [25] otel_0.2.0        viridisLite_0.4.3 cli_3.6.6
#> [28] withr_3.0.2       magrittr_2.0.4   ps_1.9.1
#> [31] processx_3.8.6    digest_0.6.39    grid_4.5.2
#> [34] rstudioapi_0.18.0 lifecycle_1.0.5  RcppArmadillo_15.2.3-1
#> [37] StanHeaders_2.32.10 vctrs_0.7.1     evaluate_1.0.5
#> [40] glue_1.8.0        farver_2.1.2     QuickJSR_1.9.0
#> [43] codetools_0.2-20  stats4_4.5.2     pkgbuild_1.4.8
#> [46] rmarkdown_2.30    pkgconfig_2.0.3  matrixStats_1.5.0
#> [49] tools_4.5.2       loo_2.9.0        htmltools_0.5.9

```